

Operating System Structures

10 Questions & Model Answers

Group 4 | 2025/2026 Academic Session

Submitted to: Dr. Oluwatosin

Bells University of Technology, Ota | CSC 308 – Operating Systems

Question 1

Distinguish between a Simple Structure and a Monolithic Structure in operating system design. Why is this distinction significant for system reliability?

Answer

Both structures place all OS functionality within a single executable, but the key distinction lies in their degree of internal organization.

Simple Structure (e.g., MS-DOS):

There are no formally defined interfaces or boundaries between subsystems. Any component can directly call any other, and application programs can even access hardware registers directly, bypassing the OS entirely. There is no separation of concerns whatsoever.

Monolithic Structure (e.g., original UNIX):

All OS services still run in one kernel binary in privileged mode, but the system defines internal interfaces between subsystems. The process scheduler, memory manager, and file system are distinct subsystems with defined responsibilities, even though they share the same address space.

Significance for reliability:

In the Simple Structure, a bug anywhere produces completely unpredictable effects because there are no boundaries. In the Monolithic Structure, defined internal interfaces constrain how components interact, making faults more predictable and traceable. However, both structures share the fundamental reliability weakness that a bug in any component — including a minor device driver — can corrupt the entire kernel and crash the system.

Question 2

What are the two categories of operating system design requirements? Explain each and give two examples of goals that fall under each category.

Answer

OS design requirements fall into two categories:

1. User Goals:

Requirements from the perspective of people who use the system. Users want the OS to be convenient to use, easy to learn, reliable, safe, and fast. Examples: The system should respond quickly to user input (performance); the system should not crash during normal use (reliability).

2. System Goals:

Requirements from the perspective of designers, implementers, and maintainers. The OS should be easy to design, implement, and maintain; flexible; efficient; and error-free. Examples: The OS should be portable across different hardware platforms without major rewriting (portability); new hardware support should be addable without restructuring the entire kernel (extensibility).

These two categories frequently conflict — maximising user convenience (more features) increases system complexity, making the OS harder to implement and maintain correctly. This is why the separation of mechanism (how something is done) from policy (what should be done) is a fundamental OS design principle — it allows policies to change without redesigning core mechanisms.

Question 3

Explain the mechanism by which a system call transfers control from user mode to kernel mode. What happens at the CPU level during this transition?

Answer

A system call is the controlled gateway through which a user-mode process requests a privileged OS service.

Step-by-step mechanism:

1. The user program calls a library wrapper function (e.g., `read()` in C). The wrapper places the system call number in a designated CPU register (e.g., RAX on x86-64) and arguments in other registers.
2. The wrapper executes a privileged trap instruction — `syscall` on x86-64, `SVC` on ARM. This triggers a CPU mode switch from user mode (Ring 3) to kernel mode (Ring 0).
3. The CPU automatically saves the current program counter, stack pointer, and CPU flags onto the kernel stack, then transfers control to the OS system call dispatcher.
4. The dispatcher uses the system call number to index into the system call table, locating and invoking the correct kernel function. Arguments are validated before use.
5. The kernel executes the requested service with full hardware access.
6. On completion, the kernel places the return value in RAX and executes `sysret`, which switches the CPU back to user mode and resumes the process at the instruction following the trap.

The critical security property of this mechanism is that user code can never jump into arbitrary kernel code — the only valid entry point is the system call interface, which validates all inputs before acting on them.

Question 4

Compare the Microkernel and Modular structures. Under what circumstances would each be the more appropriate design choice?

Answer

Microkernel Structure:

Keeps only the absolute minimum in kernel space — basic IPC, elementary memory management, and low-level CPU scheduling. All other services (file systems, device drivers,

network stacks) run as user-space server processes communicating via message passing. Example: Mach, QNX.

Modular Structure:

Maintains a small essential kernel core but adds functionality through dynamically loadable kernel modules (LKMs) that run in kernel mode and communicate via direct function calls — no message passing overhead. Example: Linux, Solaris.

Key differences:

Performance: Modular is faster — kernel-mode modules use direct function calls. Microkernel has IPC overhead for every service interaction.

Fault isolation: Microkernel is stronger — a crashed user-space server does not bring down the kernel. A faulty module in the modular structure runs in kernel mode and can still crash the system.

Appropriate circumstances:

Choose the Microkernel when reliability and fault isolation are paramount — safety-critical embedded systems (medical devices, avionics, automotive), where a crashed OS component must never take down the whole system. Choose the Modular structure for general-purpose computing where performance matters and broad hardware support must be managed flexibly — as Linux demonstrates across servers, desktops, and Android devices.

Question 5

Identify and explain the three components of the XNU kernel used in macOS and iOS. What structural model does each component represent?

Answer

XNU (X is Not Unix) integrates three major components, each representing a different structural model:

1. Mach Microkernel:

Provides the fundamental OS abstractions: tasks (address spaces), threads (units of execution), ports (communication endpoints), and messages (IPC data). Also handles virtual memory management, offering a unified memory model that supports file-backed and anonymous mappings with efficient copy-on-write sharing between processes. Structural model: Microkernel — minimal kernel, prioritising reliability and fault isolation.

2. BSD Subsystem:

Derived from FreeBSD. Provides POSIX-compatible system calls, the networking stack (TCP/IP), the Virtual File System (VFS) layer, and standard UNIX utilities. Runs in kernel space rather than as a user-space server, achieving the performance of in-kernel code execution. Structural model: Monolithic — performance-critical services compiled directly into the kernel.

3. I/O Kit Device Driver Framework:

A C++ framework for writing device drivers. Supports drivers implemented either as user-space processes (microkernel tradition, for fault isolation) or as kernel extensions/kexts (monolithic tradition, for performance), depending on requirements. Structural model: Modular — dynamically loadable extensions added at runtime.

The combination of all three makes XNU genuinely hybrid: Mach provides the isolation foundation, BSD provides the performance layer, and I/O Kit provides extensibility — properties no single pure structure can deliver simultaneously.

Question 6

What are the two levels at which operating system services exist? List three services from each level.

Answer

Operating system services exist at two levels:

Level 1 — User-Oriented Services (provided for the benefit of users and the programs they execute):

- i. User Interface: The OS provides a CLI or GUI through which users interact with the system.
- ii. Program Execution: The OS loads programs into memory, manages their execution lifecycle, and handles normal or abnormal termination.
- iii. File-System Manipulation: The OS provides a file system abstraction — creating, reading, writing, deleting, and managing permissions on files and directories, hiding the complexity of physical storage.

Level 2 — System-Oriented Services (not directly visible to users but essential for efficient system operation):

- i. Resource Allocation: When multiple processes run concurrently, the OS allocates CPU time, memory, storage, and I/O device access fairly and efficiently.
 - ii. Accounting and Logging: The OS records resource consumption by users and processes, used for billing, performance tuning, and security auditing.
 - iii. Protection and Security: Protection mechanisms control access to resources according to defined policies; security mechanisms defend against external threats such as malware and unauthorised intrusion.
-

Question 7

Explain the Android OS architecture. Why is Android classified as a hybrid operating system?

Answer

Android is built as a multi-layered stack, with each layer drawing from a different structural model:

Layer 1 — Linux Kernel (base):

Provides process management, memory management, device drivers, and the security model. Uses the modular monolithic approach — a monolithic kernel with dynamically loadable modules for hardware support.

Layer 2 — Native C/C++ Libraries:

Libraries such as libc, OpenSSL, and SQLite provide performance-critical services to higher layers without going through the full Android framework.

Layer 3 — Android Runtime (ART):

Executes applications written in Java or Kotlin using Ahead-Of-Time (AOT) compilation to native machine code. Abstracts the underlying OS from application developers.

Layer 4 — Java API Framework:

Provides the standard Android programming interface — Activity Manager, Window Manager, Content Providers, etc.

Layer 5 — Applications:

The topmost layer of user-facing apps.

Why hybrid:

Android combines the modular monolithic model (Linux kernel), the layered model (framework and API layers), and a virtual machine abstraction (ART runtime) in a single system. No single structural paradigm describes the full Android stack — it deliberately draws the strengths of multiple approaches at different levels.

Question 8

Trace the complete sequence of events in the operating system boot process, from the moment power is applied to the point at which user applications can execute.

Answer

Stage 1 — Firmware Execution (UEFI):

On power-on, the CPU executes a program stored in ROM on the motherboard — the UEFI (Unified Extensible Firmware Interface), which replaced the older BIOS on modern systems. UEFI performs a Power-On Self-Test (POST), verifying that hardware components are present and functional. It then initialises hardware and searches for a bootable storage device.

Stage 2 — Bootloader:

UEFI loads a bootloader from the bootable device. The bootloader locates the OS kernel on storage, loads it into main memory, and transfers execution to it. On Linux systems this is typically GRUB (Grand Unified Bootloader); on Windows it is the Windows Boot Manager. The bootloader may also present a boot menu in dual-boot configurations.

Stage 3 — Kernel Initialisation:

The kernel begins executing and: initialises its internal data structures; detects and initialises all hardware devices; mounts the root file system.

Stage 4 — First User-Space Process:

The kernel launches the first user-space process — init (or systemd on modern Linux), or wininit.exe on Windows. This process is the ancestor of all subsequent processes, and is responsible for starting all required system daemons and services. Only after this stage can user applications execute.

Question 9

What is System Generation (SYSGEN)? How do modern operating systems handle the problem that SYSGEN was designed to solve?

Answer

Definition:

System Generation (SYSGEN) is the process of configuring an operating system specifically for a particular hardware installation. Different computing environments have different hardware — different CPU types, amounts of RAM, I/O devices, and network interfaces. Including support for every possible configuration in a single OS binary would be impractical and wasteful.

Historical approach:

In older mainframe-era systems, SYSGEN was a complex manual process. System administrators selected the OS components required for their specific hardware, and the OS was compiled and linked from source with only those components included.

Information required for SYSGEN includes:

The type and number of CPUs, amount and type of RAM installed, I/O devices present and their connections, file systems to be supported, and network protocols to be used.

How modern OSes handle this:

Modern operating systems solve the same problem using the modular structure. The kernel contains only a small core that runs on all supported hardware. Optional components — device drivers for specific hardware, file system implementations, network protocols — are provided as dynamically loadable modules (LKMs on Linux). At boot time, the OS automatically detects present hardware and loads only the modules required for that configuration. This eliminates the need for manual compilation while achieving the same goal of a hardware-appropriate OS.

Question 10

Why have hybrid operating system structures become dominant in modern computing? Support your answer with examples of real-world operating systems and identify any contexts where a pure structure remains preferable.

Answer

Hybrid structures dominate because every pure structural model has an irreducible fundamental limitation:

Simple/Monolithic: Fast, but a bug in any component crashes the entire system.

Layered: Clean, but performance overhead and circular dependency problems make pure implementation impractical at scale.

Microkernel: Strong fault isolation, but IPC overhead is prohibitive for general-purpose computing where OS services are called millions of times per second.

Modular: Flexible and near-monolithic in performance, but lacks the fault isolation that user-space services provide.

No single paradigm delivers performance, reliability, security, and extensibility simultaneously. Hybrid structures apply each model where it excels:

macOS/iOS (XNU):

Mach microkernel for IPC and virtual memory (isolation), BSD in kernel space for POSIX services (performance), I/O Kit modules for device drivers (flexibility).

Linux:

Monolithic core for speed, loadable modules for extensibility — a modular monolithic hybrid powering Android, cloud servers, and supercomputers.

Windows:

Small microkernel for fundamental CPU operations, a modular executive for OS services, HAL for hardware abstraction, and user-mode API DLLs — layered, microkernel, and modular concepts combined.

Where pure structures remain preferable:

Pure Microkernels (QNX, MINIX 3) remain the right choice for safety-critical embedded systems — medical devices, avionics, automotive control — where fault isolation matters more than raw performance. Simple structures remain appropriate in deeply constrained microcontroller environments (kilobytes of RAM, single dedicated task) where abstraction layers have no justification.